

Redis Data Structures & Patterns

Chapter 06 - Sub-Chapter 02

NoSQL Databases Series

What is Redis?	In-memory store -- 500x faster than disk databases
Strings	Counters, sessions, and cached values
Lists, Sets, Hashes	Queues, unique collections, structured objects
Sorted Sets + Pub/Sub	Leaderboards, rankings, real-time messaging
Caching Patterns	Cache-Aside, Write-Through, Write-Behind

1

What is Redis?

The world's fastest in-memory data store

Imagine your database is a library. Every time your app needs information, it sends a librarian to find the book, walk to the shelf, find the page, and walk back. This takes time -- maybe 5-50 milliseconds per query.

Now imagine you have a **personal assistant** who memorises the 100 most-asked questions and keeps the answers on a notepad on your desk. You get the answer in 0.1 milliseconds -- 500x faster. **That notepad is Redis.**

```
// Without Redis -- every user request hits the database
// URL: GET /api/products/bestsellers

// Java + JDBC (no cache)
public List<Product> getBestsellers() {
    // This SQL runs on EVERY request -- even if products haven't changed
    return jdbcTemplate.query(
        "SELECT p.*, SUM(oi.quantity) as sold " +
        "FROM products p " +
        "JOIN order_items oi ON p.id = oi.product_id " +
        "GROUP BY p.id ORDER BY sold DESC LIMIT 10",
        productRowMapper
    );
    // Takes 150ms, hits DB indexes, reads 50,000 rows -- on every request
    // 1000 users/minute = 1000 slow DB queries/minute -- DB gets overloaded
}

// With Redis cache -- DB hit only once every 5 minutes
public List<Product> getBestsellers() {
    String cacheKey = "bestsellers:top10";
    String cached = redis.get(cacheKey);

    if (cached != null) {
        return objectMapper.readValue(cached, listType); // 0.1ms -- from memory
    }

    // Cache miss -- go to DB (happens rarely)
    List<Product> products = fetchFromDatabase();
    redis.setex(cacheKey, 300, objectMapper.writeValueAsString(products));
    // Cache expires in 300 seconds -- auto-refreshed
    return products;
}
// 1000 users/minute -- only 1 DB query per 5 minutes. DB is free.
```

Key Rules

- Redis stores data in RAM -- reads take 0.1ms vs 5-50ms for disk-based databases.
- Redis is single-threaded for commands -- no locking, no deadlocks, very predictable.
- Data is optionally persisted to disk (RDB snapshots or AOF log) for durability.
- Redis supports 5 core data structures + advanced types (Streams, Bitmaps, HyperLogLog).
- Used by Twitter, GitHub, Stack Overflow, Airbnb as the primary caching layer.

Q1. Why is Redis so fast?

Redis stores all data in RAM (memory), not on disk. RAM access is 1000x faster than SSD. Additionally, Redis is single-threaded so there is no locking overhead.

Q2. Does Redis lose data when server restarts?

By default, yes. But Redis supports persistence: RDB (snapshot every N seconds) and AOF (append-only log of every write). Most setups use both for durability.

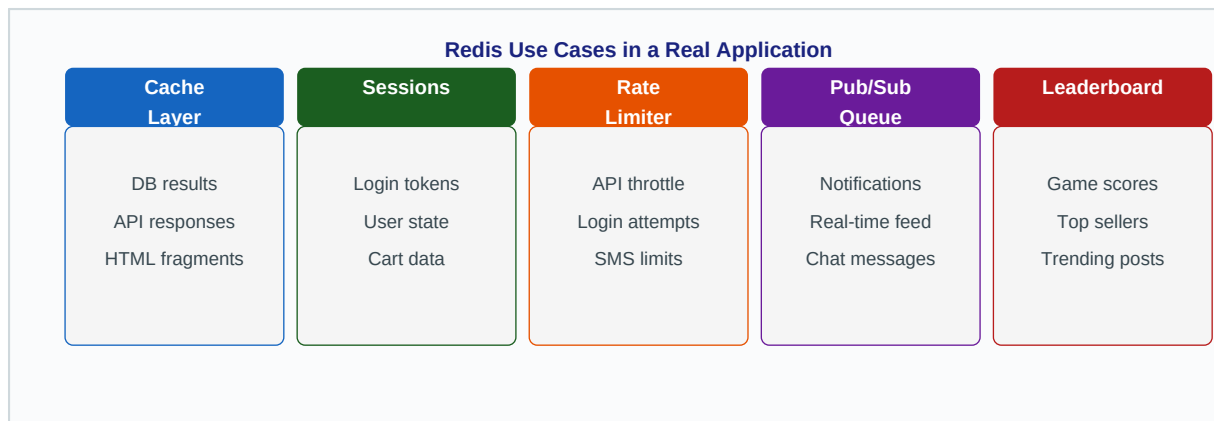
Q3. Is Redis a database or a cache?

Both. Redis can be used purely as a cache (with TTL expiry) or as a primary database for data that fits in memory. Redis Cluster supports terabytes of data.

2 Data Structure 1: Strings

Counters, sessions, and cached values

Redis Strings are the simplest structure -- a key maps to a single value. The value can be text, a number, or binary data (JSON, images, serialized objects). Do not let the name fool you -- Redis Strings are not just for text.



Five major Redis use cases -- all built on its data structures.

Use case 1 -- Cache a database result:

```
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.ValueOperations;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.time.Duration;

@Service
public class ProductService {
    @Autowired private StringRedisTemplate redis;
    @Autowired private ProductRepository repo;
    @Autowired private ObjectMapper mapper;

    public Product getProduct(Long id) throws Exception {
        String key = "product:" + id;
        ValueOperations<String, String> ops = redis.opsForValue();

        String cached = ops.get(key);
        if (cached != null) {
            return mapper.readValue(cached, Product.class); // cache hit
        }

        Product p = repo.findById(id).orElseThrow();
        ops.set(key, mapper.writeValueAsString(p), Duration.ofMinutes(10));
        return p;
        // Next 10 minutes: served from Redis (0.1ms, not DB)
    }

    // When product is updated -- invalidate cache
    public void updateProduct(Product p) {
        repo.save(p);
        redis.delete("product:" + p.getId()); // force refresh on next read
    }
}
```

Use case 2 -- Atomic counter (page views, likes):

```

import org.springframework.data.redis.core.StringRedisTemplate;

@Service
public class CounterService {
    @Autowired private StringRedisTemplate redis;

    // INCR is atomic -- safe even with 10,000 concurrent users
    public Long incrementPageView(String articleId) {
        return redis.opsForValue().increment("views:" + articleId);
        // Redis INCR is a single atomic operation -- no race condition
        // Without Redis: SELECT count, UPDATE count -- 2 operations, race condition possible
    }

    // Rate limiter -- max 100 requests per minute per user
    public boolean isRateLimited(String userId) {
        String key = "ratelimit:" + userId + ":" + System.currentTimeMillis() / 60000;
        Long count = redis.opsForValue().increment(key);
        if (count == 1) {
            redis.expire(key, Duration.ofMinutes(1)); // auto-expire after 1 minute
        }
        return count > 100; // true = rate limited
    }
}

```

Use case 3 -- User session storage:

```

import org.springframework.data.redis.core.StringRedisTemplate;
import java.util.UUID;

@Service
public class SessionService {
    @Autowired private StringRedisTemplate redis;
    @Autowired private ObjectMapper mapper;

    public String createSession(User user) throws Exception {
        String token = UUID.randomUUID().toString();
        SessionData session = new SessionData(user.getId(), user.getRole());

        // Store session -- auto-expires in 30 minutes
        redis.opsForValue().set(
            "session:" + token,
            mapper.writeValueAsString(session),
            Duration.ofMinutes(30)
        );
        return token; // return token to client (stored in cookie/header)
    }

    public SessionData getSession(String token) throws Exception {
        String data = redis.opsForValue().get("session:" + token);
        if (data == null) throw new UnauthorizedException("Session expired");
        return mapper.readValue(data, SessionData.class);
    }

    public void logout(String token) {
        redis.delete("session:" + token); // instant logout
    }
}

```

Key Rules

- SET key value EX seconds -- store with automatic expiry (TTL).
- INCR/INCRBY -- atomic counters, safe for concurrent use without locks.

- SETNX (SET if Not eXists) -- used for distributed locks.
- String values can hold up to 512 MB -- store JSON, images, serialized objects.
- MSET/MGET -- set or get multiple keys in one network round-trip.

Q1. What is TTL in Redis?

Time-To-Live -- the number of seconds before a key automatically deletes itself. Set with EX in SET command or EXPIRE command. Essential for cache and session management.

Q2. What happens if two requests INCR the same counter at the same time?

Redis handles this safely -- INCR is atomic. Both increments happen sequentially inside Redis's single thread. No race condition, no lost updates.

Q3. How do I store an object (not just a string) in Redis?

Serialize it to JSON using Jackson (objectMapper.writeValueAsString) before storing. Deserialize on read. Alternatively use Redis Hash to store individual fields.

3 Lists, Sets and Hashes

Queues, unique collections, and structured objects

Redis Data Structures at a Glance				
String	List	Set	Hash	Sorted Set
SET key val GET key INCR counter	LPUSH / RPUSH LPOP / RPOP LRANGE key 0 -1	SADD key mem SMEMBERS key SINTER s1 s2	HSET key f v HGET key f HGETALL key	ZADD key sc m ZRANK key m ZRANGE key 0 -1

Redis data structures and their core commands.

List -- for queues, recent activity, chat history:

A Redis List is like a double-ended queue -- you can push to the left or right and pop from either end. Perfect for activity feeds, task queues, and chat history.

```
import org.springframework.data.redis.core.ListOperations;
import org.springframework.data.redis.core.StringRedisTemplate;

@Service
public class ActivityFeedService {
    @Autowired private StringRedisTemplate redis;

    // User posts something -- add to followers' feeds
    public void addToFeed(String followerId, String activityJson) {
        String key = "feed:" + followerId;
        ListOperations<String, String> list = redis.opsForList();

        list.leftPush(key, activityJson);    // add to front (newest first)
        list.trim(key, 0, 99);               // keep only latest 100 items
        // Auto-trims the list -- no unbounded growth
    }

    // User opens their feed -- get latest 20 items
    public List<String> getFeed(String userId, int page) {
        int start = page * 20;
        return redis.opsForList().range("feed:" + userId, start, start + 19);
    }

    // Simple task queue -- worker pops jobs from queue
    public void enqueueEmailJob(String jobJson) {
        redis.opsForList().rightPush("queue:emails", jobJson);
    }

    public String dequeueEmailJob() {
        // BLPOP -- blocks until an item is available (no polling needed)
        return redis.opsForList().leftPop("queue:emails", Duration.ofSeconds(5));
    }
}
```

Set -- for unique members, tags, followers:

A Redis Set stores unique values with no duplicates. Powerful set operations (intersection, union, difference) work in $O(N)$ time.

```
import org.springframework.data.redis.core.SetOperations;

@Service
public class SocialGraphService {
    @Autowired private StringRedisTemplate redis;

    public void follow(String userId, String targetId) {
        // userId follows targetId
        redis.opsForSet().add("following:" + userId, targetId);
        redis.opsForSet().add("followers:" + targetId, userId);
    }

    public void unfollow(String userId, String targetId) {
        redis.opsForSet().remove("following:" + userId, targetId);
        redis.opsForSet().remove("followers:" + targetId, userId);
    }

    // "People you may know" -- friends of friends
    // Find users that both Manish and Priya follow
    public Set<String> mutualFollowing(String user1, String user2) {
        return redis.opsForSet().intersect(
            "following:" + user1,
            "following:" + user2
        );
    }

    // Tag-based product filtering -- products tagged both "sale" AND "electronics"
    public Set<String> productsInBothTags(String tag1, String tag2) {
        return redis.opsForSet().intersect(
            "tag:" + tag1,
            "tag:" + tag2
        );
    }
}
```

Hash -- for structured objects (user profiles, product data):

A Redis Hash stores a map of field-value pairs under one key -- like a row in a database table. More memory-efficient than storing JSON because you can update individual fields without re-serializing the whole object.

```
import org.springframework.data.redis.core.HashOperations;

@Service
public class UserProfileCache {
    @Autowired private StringRedisTemplate redis;

    public void cacheUserProfile(User user) {
        String key = "user:" + user.getId();
        HashOperations<String, String, String> hash = redis.opsForHash();

        Map<String, String> fields = new HashMap<>();
        fields.put("name", user.getName());
        fields.put("email", user.getEmail());
        fields.put("role", user.getRole());
        fields.put("plan", user.getPlan());
        hash.putAll(key, fields);
    }
}
```



```

        redis.expire(key, Duration.ofHours(1));
    }

    // Update ONLY the plan field -- no need to reload entire object
    public void updatePlan(String userId, String newPlan) {
        redis.opsForHash().put("user:" + userId, "plan", newPlan);
        // With JSON string: would have to GET, deserialize, update, re-serialize, SET
        // With Hash: single HSET command -- much more efficient
    }

    public String getUsername(String userId) {
        return (String) redis.opsForHash().get("user:" + userId, "name");
    }

    public Map<Object, Object> getFullProfile(String userId) {
        return redis.opsForHash().entries("user:" + userId);
    }
}

```

Key Rules

- List: use for queues, feeds, and chat history. BLPOP blocks until data arrives -- no polling.
- Set: use for unique membership, tags, and social graphs. SINTERSTORE for computed intersections.
- Hash: use for structured objects. Update individual fields without touching the whole object.
- All List/Set/Hash operations are $O(1)$ or $O(N)$ -- predictable performance.
- Redis Sets support SUNION, SINTER, SDIFF -- powerful for recommendation engines.

Q1. When should I use Hash vs serialized JSON String?

Use Hash when you need to update individual fields frequently (e.g., user online status). Use JSON String when you always read/write the whole object at once.

Q2. What is BLPOP and why use it?

Blocking left-pop -- the client waits (blocks) until an item appears in the list. Used in task queues so workers do not need to poll Redis every second.

Q3. Can a Redis Set tell me if a user is following another?

Yes -- SISMEMBER following:userId targetId returns 1 (yes) or 0 (no) in $O(1)$ time, regardless of how many users they follow.

4 Sorted Sets and Pub/Sub

Leaderboards, rankings, and real-time messaging

Sorted Set -- leaderboards, priority queues, trending topics:

A Sorted Set is like a Set but every member has a **score** (a floating point number). Members are always sorted by score. This makes it perfect for leaderboards, priority queues, and time-windowed analytics.

```
import org.springframework.data.redis.core.ZSetOperations;
import org.springframework.data.redis.core.ZSetOperations.TypedTuple;

@Service
public class LeaderboardService {
    @Autowired private StringRedisTemplate redis;

    // Player finishes a game -- update their score
    public void addScore(String playerId, double score) {
        // ZADD -- adds member with score, or updates if member exists
        redis.opsForZSet().add("leaderboard:global", playerId, score);
    }

    // Increment score (add to existing -- e.g., points earned this round)
    public void incrementScore(String playerId, double points) {
        redis.opsForZSet().incrementScore("leaderboard:global", playerId, points);
    }

    // Get top 10 players with their scores (highest first)
    public Set<TypedTuple<String>> getTop10() {
        return redis.opsForZSet().reverseRangeWithScores(
            "leaderboard:global", 0, 9
        );
        // Returns: [{Manish, 98500.0}, {Priya, 87300.0}, ...]
    }

    // Get a player's rank (0-indexed, lower = better)
    public Long getPlayerRank(String playerId) {
        Long rank = redis.opsForZSet().reverseRank("leaderboard:global", playerId);
        return rank != null ? rank + 1 : null; // 1-indexed for display
    }

    // Players ranked 50-100 (pagination)
    public Set<TypedTuple<String>> getPage(int page) {
        long start = (long) page * 50;
        return redis.opsForZSet().reverseRangeWithScores(
            "leaderboard:global", start, start + 49
        );
    }

    // Trending topics -- tweets in last hour
    // Score = timestamp, range query = "last hour"
    public void recordTweet(String hashtag) {
        double now = System.currentTimeMillis();
        redis.opsForZSet().add("trending", hashtag, now);
        // Remove tweets older than 1 hour
        redis.opsForZSet().removeRangeByScore(
            "trending", 0, now - 3_600_000
        );
    }

    public Set<String> getTrending() {
        double now = System.currentTimeMillis();
    }
```

```

        return redis.opsForZSet().rangeByScore(
            "trending", now - 3_600_000, now
        );
    }
}

```

Pub/Sub -- real-time notifications and messaging:

Redis Pub/Sub lets one component publish a message to a channel and all subscribers receive it instantly. Unlike a queue, messages are fire-and-forget -- if no subscriber is listening, the message is lost.

```

import org.springframework.data.redis.connection.MessageListener;
import org.springframework.data.redis.listener.ChannelTopic;
import org.springframework.data.redis.listener.RedisMessageListenerContainer;
import org.springframework.data.redis.core.StringRedisTemplate;

// Publisher -- sends notification when order is placed
@Service
public class OrderService {
    @Autowired private StringRedisTemplate redis;

    public Order placeOrder(Order order) {
        Order saved = orderRepo.save(order);

        // Notify all listeners that an order was placed
        String msg = objectMapper.writeValueAsString(saved);
        redis.convertAndSend("channel:order_placed", msg);
        // All subscribers on this channel receive msg instantly
        return saved;
    }
}

// Subscriber 1 -- sends confirmation email
@Component
public class EmailNotificationListener implements MessageListener {
    @Override
    public void onMessage(Message message, byte[] pattern) {
        String orderJson = new String(message.getBody());
        Order order = objectMapper.readValue(orderJson, Order.class);
        emailService.sendOrderConfirmation(order);
    }
}

// Subscriber 2 -- updates inventory
@Component
public class InventoryListener implements MessageListener {
    @Override
    public void onMessage(Message message, byte[] pattern) {
        Order order = objectMapper.readValue(...);
        inventoryService.reduceStock(order.getItems());
    }
}

// Register both listeners on the same channel
@Configuration
public class RedisConfig {
    @Bean
    public RedisMessageListenerContainer container(
        RedisConnectionFactory factory,
        EmailNotificationListener emailListener,
        InventoryListener inventoryListener) {

```

```
RedisMessageListenerContainer container = new RedisMessageListenerContainer();
container.setConnectionFactory(factory);
container.addMessageListener(emailListener, new ChannelTopic("channel:order_placed"));
container.addMessageListener(inventoryListener, new ChannelTopic("channel:order_placed"));
return container;
}
}
```

Key Rules

- Sorted Set: ZADD adds with score, ZREVRANGE returns top-N, ZRANK gives position -- all $O(\log N)$.
- Use Sorted Sets for leaderboards, priority queues, and sliding window analytics.
- Pub/Sub: PUBLISH sends to channel, SUBSCRIBE listens -- no persistence, fire-and-forget.
- For reliable messaging (no lost messages), use Redis Streams instead of Pub/Sub.
- Sorted Set scores can be timestamps -- enables efficient time-based range queries.

Q1. What is the time complexity of Sorted Set operations?

ZADD, ZRANK, ZSCORE are $O(\log N)$. ZRANGE is $O(\log N + M)$ where M is the number of elements returned. All very fast even with millions of members.

Q2. What is the difference between Redis Pub/Sub and a message queue?

Pub/Sub is fire-and-forget -- if no subscriber is listening, the message is lost. A queue (like RabbitMQ or Redis Streams) persists messages until a consumer reads them.

Q3. Can I use Sorted Sets for a priority queue?

Yes -- ZADD with priority as score. ZPOPMIN removes and returns the item with the lowest score (highest priority if you use lower numbers for higher priority).

5 Caching Patterns

Cache-Aside, Write-Through, Write-Behind

There are three main strategies for keeping your cache in sync with your database. Choosing the wrong one causes either stale data or data loss.

Pattern 1 -- Cache-Aside (Lazy Loading) -- most common:

Application checks cache first. On miss, loads from DB and populates cache. Cache only contains data that was actually requested.

```
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.CachePut;

// Spring @Cacheable -- Cache-Aside with one annotation
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")
    // Spring checks Redis for "products::42" before calling the method
    // Cache hit -> returns cached value, method body NEVER runs
    // Cache miss -> runs method, stores result in Redis automatically
    public Product getProduct(Long id) {
        return productRepo.findById(id).orElseThrow(); // only runs on cache miss
    }

    @CachePut(value = "products", key = "#product.id")
    // Always runs method AND updates cache -- use after update
    public Product updateProduct(Product product) {
        return productRepo.save(product);
    }

    @CacheEvict(value = "products", key = "#id")
    // Runs method AND removes from cache -- use after delete
    public void deleteProduct(Long id) {
        productRepo.deleteById(id);
    }

    @CacheEvict(value = "products", allEntries = true)
    // Clear entire cache -- use after bulk operations
    public void refreshAllProducts() {
        // This will force fresh DB reads for all products
    }
}

// application.yml configuration:
// spring:
//   cache:
//     type: redis
//   redis:
//     host: localhost
//     port: 6379
//     time-to-live: 600000 # 10 minutes in milliseconds
```

Pattern 2 -- Write-Through -- always write to cache AND DB:

```
// Write-Through: every write goes to BOTH Redis and DB simultaneously
// Advantage: cache is never stale
// Disadvantage: every write is slower (two operations)
```

```

@Service
public class UserService {
    @Autowired private StringRedisTemplate redis;
    @Autowired private UserRepository repo;
    @Autowired private ObjectMapper mapper;

    public User createUser(User user) throws Exception {
        // Write to DB first
        User saved = repo.save(user);

        // Immediately write to cache
        redis.opsForValue().set(
            "user:" + saved.getId(),
            mapper.writeValueAsString(saved),
            Duration.ofHours(24)
        );
        // Next read is always a cache hit -- no stale data
        return saved;
    }
}

```

Pattern 3 -- Write-Behind (Write-Back) -- async DB writes:

```

// Write-Behind: write to cache immediately, DB update happens async (later)
// Advantage: extremely fast writes
// Disadvantage: risk of data loss if Redis crashes before DB is updated
// Use for: non-critical counters, analytics, like counts

```

```

@Service
public class LikeService {
    @Autowired private StringRedisTemplate redis;
    @Autowired private LikeRepository repo;

    // Write to Redis immediately -- user sees instant feedback
    public void likePost(String postId, String userId) {
        redis.opsForSet().add("likes:" + postId, userId);
        redis.opsForValue().increment("likecount:" + postId);
        // Return immediately -- DB write happens in background
    }

    // Scheduled job runs every 60 seconds -- flushes Redis to DB
    @Scheduled(fixedDelay = 60_000)
    public void flushLikesToDB() {
        // Read all pending like counts from Redis and write to DB
        Set<String> keys = redis.keys("likecount:*");
        for (String key : keys) {
            String postId = key.replace("likecount:", "");
            Long count = Long.parseLong(redis.opsForValue().get(key));
            repo.updateLikeCount(postId, count);
        }
    }
}

```

Pattern	Read speed	Write speed	Stale risk	Data loss risk	Use for
Cache-Aside	Fast after first miss	Normal (DB only)	Low (TTL controlled)	None	Most read-heavy APIs
Write-Through	Always fast	Slower (DB+cache)	None	None	User profiles, settings

Write-Behind	Always fast	Fastest (cache only)	Low	If crash before flush	Counters, analytics
--------------	-------------	----------------------	-----	-----------------------	---------------------

Key Rules

- Cache-Aside is the default choice -- simple, safe, works for 80% of use cases.
- Always set a TTL -- never cache indefinitely unless data truly never changes.
- Cache invalidation is the hardest problem -- prefer short TTLs over complex invalidation logic.
- Write-Through guarantees no stale reads but adds latency to every write.
- Write-Behind is only for data where slight loss is acceptable (view counts, analytics).

Q1. What is a cache stampede and how to prevent it?

When a cached item expires, many requests simultaneously hit the DB to refresh it. Prevent with: mutex locks (only one thread refreshes), probabilistic early expiry, or background refresh before TTL expires.

Q2. How do I handle cache invalidation across microservices?

Publish an event (via Redis Pub/Sub or Kafka) when data changes. Each service that cached that data listens and deletes its cached copy. Or use short TTLs and accept brief staleness.

Q3. What is the difference between cache eviction and cache expiry?

Expiry: key deletes itself after TTL. Eviction: Redis removes keys to free memory when memory limit is reached. Configure eviction policy (LRU, LFU) in redis.conf.